

LAPORAN DATA MINING

Deteksi Fraud dan Segmentasi Transaksi pada Dataset PaySim

(6.362.604 transaksi, mobile-money banking simulation)

Ringkasan Temuan dan Metodologi — Phase 1 s.d. Phase 4

Ringkasan Eksekutif

Proyek ini menganalisis dataset PaySim (simulasi transaksi mobile-money) berjumlah 6.362.604 transaksi setelah pembersihan data, dengan 8.197 di antaranya berstatus fraud (0,13%). Laporan ini merangkum temuan, metodologi, dan kode yang benar-benar digunakan pada setiap tahap (Phase 1 sampai Phase 4), sebagai dasar penyusunan dashboard dan Knowledge Discovery Report pada Phase 5.

- Phase 1 (Preprocessing): Data terbukti bersih secara struktural. Sebelas temuan EDA menunjukkan fraud hanya terjadi pada TRANSFER dan CASH_OUT, tidak pernah melibatkan merchant, dan dua fitur turunan (origError/destError dan origDrainedToZero) terbukti berkaitan kuat dengan fraud sejak tahap eksplorasi.
- Phase 2 (Clustering): Empat algoritma diuji. K-Means (K=4) dipilih sebagai acuan segmentasi karena mencakup 100% data dan tervalidasi Davies-Bouldin Index, menghasilkan 4 segmen transaksi yang bisa diberi nama bisnis. Kombinasi BIRCH + HDBSCAN dipilih sebagai metode deteksi outlier akhir, menekan proporsi outlier dari 71% (HDBSCAN langsung) menjadi 0,06%, dengan fraud rate 16,93% pada kelompok outlier tersebut (131x baseline).
- Phase 3 (Association Rules): Pola "akun dikuras habis + saldo konsisten secara matematis" ditemukan sebagai sinyal fraud terkuat (lift 776, confidence 100%). Sepuluh rule teratas seluruhnya didukung metrik support/confidence/lift yang jelas dan diberi interpretasi bisnis.
- Phase 4 (Anomaly Detection): Skema skor risiko berbobot (IQR, Z-Score, Isolation Forest, Balance Mismatch, dan outlier BIRCH+HDBSCAN) berhasil menangkap 17,43% dari seluruh fraud dengan enrichment 17,4x baseline. Cross-reference dengan segmen K-Means mengungkap satu segmen kecil (Segmen 4) yang sangat menyimpang secara statistik namun ternyata bukan fraud - contoh nyata pentingnya membedakan anomali dari fraud.

Phase 1 — Data Preprocessing dan Exploratory Data Analysis (EDA)

Penanggung jawab: Data Engineer. Dataset yang digunakan adalah PaySim, simulasi transaksi perbankan mobile-money, dengan 6.362.620 baris dan 11 kolom sebelum dibersihkan.

1.1 Gambaran Umum Dataset

```
df = pd.read_csv('../datasets/paysim-dataset.csv')
df.info()
df.shape
```

Output

```
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
  step (int64), type (object), amount (float64), nameOrig (object),
  oldbalanceOrg (float64), newbalanceOrg (float64), nameDest (object),
  oldbalanceDest (float64), newbalanceDest (float64),
  isFraud (int64), isFlaggedFraud (int64)

Shape: (6362620, 11)
```

Pengecekan missing value (`df.isnull().sum()`) menunjukkan tidak ada nilai kosong sama sekali di seluruh 11 kolom. Tipe data juga sudah konsisten: step dan flag fraud sebagai integer, amount dan seluruh kolom saldo sebagai float, sementara type, nameOrig, dan nameDest sebagai string.

1.2 Sebelas Temuan Utama EDA

Setiap temuan berikut diverifikasi langsung dari kode dan output notebook Phase 1 (bukan asumsi), sehingga bisa dipertanggungjawabkan sebagai bagian dari Preprocessing Report.

1. Tidak ada transaksi ke diri sendiri (self-transaction)

```
self_transaction = df[df['nameOrig'] == df['nameDest']]
print(f"Jumlah: {len(self_transaction)}")
```

Output

```
Jumlah: 0
```

Temuan: Tidak ditemukan satu pun transaksi di mana nameOrig sama dengan nameDest. Setiap transaksi selalu melibatkan dua akun yang berbeda, sesuai dengan pola bisnis normal transaksi keuangan.

2. Tidak ada baris duplikat persis

```
df.duplicated().sum()
# Cek manual tambahan pada kombinasi kolom kunci
dup_cols = ['step', 'type', 'nameOrig', 'nameDest', 'oldbalanceOrg']
duplicates = df[df.duplicated(subset=dup_cols, keep=False)]
print(f"Jumlah baris duplikat: {len(duplicates)}")
```

Output

```
df.duplicated().sum() -> 0
Jumlah baris duplikat (subset kolom kunci): 0
```

Temuan: Tidak ditemukan baris duplikat, baik dengan pengecekan bawaan pandas (seluruh kolom identik) maupun pengecekan manual pada kombinasi step, type, nameOrig, nameDest, dan oldbalanceOrg. Setiap baris transaksi bersifat unik.

3. Tidak ada tabrakan (collision) ID akun antara customer dan merchant

```
print(df['nameOrig'].str[0].value_counts())
print(df['nameDest'].str[0].value_counts())
```

Output

```
nameOrig
C    6362620

nameDest
C    4211125
M    2151495
```

Temuan: Seluruh akun asal (nameOrig) menggunakan prefiks "C" (Customer) tanpa kecuali. Akun tujuan (nameDest) terdiri dari campuran "C" (Customer, 4.211.125 baris) dan "M" (Merchant, 2.151.495 baris). Tidak ada prefiks lain yang muncul, sehingga struktur ID akun konsisten dengan desain dataset PaySim: transaksi selalu berasal dari customer, dan bisa berakhir di customer lain atau di merchant.

4. Tidak ada nilai negatif pada amount maupun kolom saldo

```
neg_check = df[(df['amount']<0) | (df['oldbalanceOrg']<0) | (df['newbalanceOrig']<0)
               | (df['oldbalanceDest']<0) | (df['newbalanceDest']<0)]
print(len(neg_check))
```

Output

```
0
```

Temuan: Tidak ditemukan nilai negatif pada amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, maupun newbalanceDest. Ini menunjukkan data mengikuti aturan bisnis yang wajar — saldo dan nominal transaksi tidak pernah bernilai minus.

5. Step yang hilang hanya step 744, itu pun karena akhir periode perekaman

```
missing_steps = set(range(1,745)) - set(df['step'].unique())
print(f"Step yang hilang: {missing_steps}")
```

Output

```
Step yang hilang: {744}
```

Temuan: step berjalan kontinu dari 1 sampai 743 (1 step merepresentasikan 1 jam), dan satu-satunya step yang tidak muncul adalah step 744 — yaitu satu jam terakhir yang memang berada di luar jendela perekaman data (total periode: 743 jam $\approx$ 31 hari). Ini bukan data hilang/rusak, melainkan batas akhir simulasi.

6. Akun tujuan bertipe merchant selalu memiliki saldo oldbalanceDest dan newbalanceDest = 0

```
print(df[df['nameDest'].str.startswith('M')]['oldbalanceDest'])
# seluruh 2.151.495 baris bernilai 0.0
relevant_types = df[df['type'].isin(['PAYMENT', 'TRANSFER', 'CASH_OUT'])]
stuck_balance = relevant_types[relevant_types['oldbalanceOrg'] == 0]
relevant_types['newbalanceOrig']
print((df['type'] == 'PAYMENT').sum())
```

Output

oldbalanceDest untuk seluruh baris nameDest berprefiks 'M': 0.0 (2.151.495 baris)
Jumlah baris type == 'PAYMENT': 2.151.495 <- match persis dengan jumlah akun merchant

Temuan: Setiap akun tujuan berprefiks M (merchant) memiliki oldbalanceDest dan newbalanceDest tetap 0, dan jumlahnya (2.151.495) sama persis dengan jumlah transaksi PAYMENT. Ini adalah karakteristik desain simulator PaySim — merchant memang tidak memiliki saldo yang tercatat — sehingga tidak boleh diperlakukan sebagai data hilang atau error.

7. Ketidaksesuaian saldo tujuan (newbalanceDest ≠ oldbalanceDest ± amount) menangkap sekitar 56% transaksi fraud

```
df['expected_new_dest'] = df['oldbalanceDest'] + df['amount']
df['diff_dest'] = df['newbalanceDest'] - df['expected_new_dest']
anomali_dest = df[df['diff_dest'].abs() > 0.01]
print(f"Anomali balance dest: {len(anomali_dest)} ({len(anomali_dest)/len(df)*100:.2f}%)")
print(pd.crosstab(anomali_dest['type'], anomali_dest['isFraud']))
```

Output

Anomali balance dest: 4188647 (65.83%)

isFraud	0	1
type		
CASH_IN	1399284	0
CASH_OUT	506881	545
DEBIT	9631	0
PAYMENT	2151495	0
TRANSFER	116716	4095

(545 + 4095) / 8213 total fraud = 56,50%

Temuan: Ketidacocokan aritmatika saldo tujuan (newbalanceDest tidak sama dengan oldbalanceDest + amount) memang terjadi pada mayoritas baris (65,83%) — ini bukan sinyal fraud secara umum, melainkan karakteristik desain data (mirip temuan poin 6). Namun jika difokuskan hanya pada baris fraud, pola ini menangkap 4.640 dari 8.213 kasus fraud (~56%), menjadikannya kandidat fitur yang menjanjikan untuk deteksi fraud — inilah dasar dibuatnya fitur destError pada tahap feature engineering.

8. Hanya dua transaksi non-fraud dengan saldo asal justru naik setelah bertransaksi

```
orig_increased = df[(df['newbalanceOrig'] > df['oldbalanceOrig']) & (df['amount'] > 0)]
```

```
orig_increased_no_cashin = orig_increased[orig_increased['type'] != 'CASH_IN']
print(len(orig_increased_no_cashin))
print(orig_increased_no_cashin['type'].value_counts())
print(orig_increased_no_cashin['isFraud'].value_counts())
```

Output

```
2
type: CASH_OUT      2
isFraud: 0          2
```

Temuan: Di luar transaksi CASH_IN (yang secara wajar meningkatkan saldo penerima dana), hanya ditemukan 2 baris di mana saldo akun pengirim (origin) justru naik setelah bertransaksi, dan keduanya berjenis CASH_OUT serta berstatus non-fraud. Karena jumlahnya sangat kecil, kedua baris ini layak dicatat sebagai kasus unik untuk investigasi lanjutan, namun tidak cukup material untuk mempengaruhi keputusan pembersihan data secara keseluruhan.

9. Sekitar 44.385 transaksi non-CASH_IN dengan saldo tujuan justru menurun setelah menerima dana

```
dest_decreased = df[(df['newbalanceDest'] < df['oldbalanceDest']) & (df['amount'] > 0)]
dest_decreased_no_cashin = dest_decreased[dest_decreased['type'] != 'CASH_IN']
print(len(dest_decreased_no_cashin))
print(dest_decreased_no_cashin['type'].value_counts())
print(dest_decreased_no_cashin['isFraud'].value_counts())
```

Output

```
44385
type
CASH_OUT      39020
TRANSFER      3931
DEBIT          1434

isFraud
0      44358
1         27
```

Temuan: Ditemukan 44.385 transaksi (di luar CASH_IN) di mana saldo akun tujuan justru berkurang meskipun akun tersebut seharusnya menerima dana — mayoritas pada CASH_OUT (39.020) dan TRANSFER (3.931). Namun hanya 27 dari baris ini (0,06%) yang berstatus fraud, sehingga pola ini kemungkinan besar lebih mencerminkan karakteristik pencatatan simulator (misalnya urutan pemrosesan saldo) dibanding sinyal fraud yang kuat, dan perlu tetap diwaspadai sebagai potensi noise, bukan dihapus begitu saja.

10. Fraud hanya terjadi pada tipe transaksi CASH_OUT dan TRANSFER

```
df.groupby("type")["isFraud"].agg(["sum", "count", "mean"])
```

Output

type	sum	count	mean
------	-----	-------	------

CASH_IN	0	1399284	0.000000
CASH_OUT	4116	2237500	0.001840
DEBIT	0	41432	0.000000
PAYMENT	0	2151495	0.000000
TRANSFER	4097	532909	0.007688

Temuan: Tidak ada satu pun kasus fraud pada CASH_IN, DEBIT, atau PAYMENT. Seluruh 8.213 kasus fraud terbagi antara CASH_OUT (4.116) dan TRANSFER (4.097) — dan menariknya, tingkat fraud pada TRANSFER (0,77%) lebih dari 4 kali lipat CASH_OUT (0,18%), meski jumlah kasusnya hampir sama. Ini menjadi dasar kuat untuk memfokuskan analisis lanjutan (Phase 2–4) pada kedua tipe transaksi ini.

11. Merchant tidak pernah menjadi tujuan transaksi fraud

```
print(df[df['nameDest'].str.startswith('M')]['isFraud'].value_counts())
print(f"nameOrig unique prefixes : {df['nameOrig'].str[0].unique()}")
```

Output

```
isFraud
0      2151495
```

```
nameOrig unique prefixes : ['C']
```

Temuan: Seluruh 2.151.495 transaksi yang menuju akun merchant berstatus non-fraud (isFraud selalu 0), sementara akun asal selalu berupa customer. Ini menunjukkan bahwa aktivitas fraud dalam dataset ini murni terjadi antar-akun customer, dan tidak pernah melibatkan merchant sebagai pihak penerima.

1.3 Data Cleaning

Berdasarkan sebelas temuan di atas, langkah pembersihan data yang benar-benar diterapkan adalah sebagai berikut:

```
cleaning_log = []

# 1) Hapus duplikat (defensif, walau hasil pengecekan = 0 baris)
before = len(df)
df.drop_duplicates(inplace=True)
removed = before - len(df)
cleaning_log.append(f"Duplicate rows removed : {removed}")

# 2) Hapus baris amount = 0
before = len(df)
df = df[df["amount"] > 0].copy()
removed = before - len(df)
cleaning_log.append(f"Zero-amount rows removed : {removed} (all were isFraud=1 CASH_OUT simulator artifacts)")

# 3) Hapus isFlaggedFraud (label bawaan simulator yang nyaris tidak berguna)
df.drop(columns=["isFlaggedFraud"], inplace=True)
```

```
cleaning_log.append("Dropped 'isFlaggedFraud' : 0.19% recall, subset of isFraud - not
predictive")

# 4) Hapus nameOrig (entropy nol, seluruhnya berprefiks 'C')
df.drop(columns=["nameOrig"], inplace=True)
cleaning_log.append("Dropped 'nameOrig' : zero entropy (all 'C' prefix), transaction ID
only")
```

Output

```
Duplicate rows removed: 0
Zero-amount rows removed: 16
```

Dua puluh baris pertama sebenarnya adalah 0 baris duplikat (sesuai temuan 2), sehingga `drop_duplicates()` hanya bersifat defensif/dokumentasi. Yang benar-benar mengurangi jumlah baris adalah penghapusan 16 baris dengan `amount = 0` — seluruhnya berlabel `isFraud = 1` pada transaksi `CASH_OUT`, kemungkinan artefak dari mekanisme simulator PaySim, bukan transaksi riil. Setelah pembersihan, dataset final berjumlah 6.362.604 baris (6.362.620 – 16).

Kolom `isFlaggedFraud` (label bawaan simulator PaySim) juga dibuang karena praktis tidak berguna — dari seluruh kasus fraud yang ada, kolom ini hanya menandai sebagian sangat kecil ($\text{recall} \approx 0,19\%$), sehingga tidak layak dijadikan fitur maupun label acuan.

1.4 Feature Engineering

Tiga fitur turunan dibuat berdasarkan temuan EDA di atas:

```
# origError / destError: seberapa jauh saldo aktual menyimpang dari yang seharusnya
# (rumus ini identik dengan pengecekan anomali saldo pada temuan #7)
df["origError"] = (df["oldbalanceOrg"] - df["amount"] - df["newbalanceOrig"]).abs()
df["destError"] = (df["oldbalanceDest"] + df["amount"] - df["newbalanceDest"]).abs()

# isDestMerchant: menandai apakah tujuan transaksi adalah merchant (berdasarkan temuan
#3 & #6)
df["isDestMerchant"] = (df["nameDest"].str.startswith("M")).astype(np.int8)
df.drop(columns=["nameDest"], inplace=True)

# origDrainedToZero: menandai apakah akun pengirim dikuras habis
df["origDrainedToZero"] = ((df["oldbalanceOrg"] > 0) & (df["newbalanceOrig"] ==
0)).astype(np.int8)
df.groupby("origDrainedToZero")["isFraud"].mean()
```

Output

```
origDrainedToZero
0      0.000038
1      0.005269
Name: isFraud, dtype: float64
```

Temuan: Akun yang dikuras habis (`origDrainedToZero = 1`) memiliki tingkat fraud rata-rata 0,53% — sekitar 139 kali lipat lebih tinggi dibanding akun yang tidak dikuras habis (0,0038%). Ini adalah salah satu sinyal biner paling kuat yang ditemukan di Phase 1, dan terbukti menjadi fitur penting di Phase 2–4.

1.5 Entropy Analysis

Entropy dihitung untuk mengukur seberapa acak/bervariasi suatu kolom (dengan pembagian 20 bin untuk kolom numerik). Nilai entropy rendah menunjukkan data cenderung seragam, sedangkan nilai tinggi menunjukkan variasi yang besar.

```
from scipy.stats import entropy as
scipy_entropy
numeric_cols = ['amount', 'oldbalanceOrig',
                'newbalanceOrig',
                'oldbalanceDest',
                'newbalanceDest', 'isFraud', 'step']
def discrete_entropy(series):
    counts =
series.value_counts(normalize=True)
    return scipy_entropy(counts)
for col in numeric_cols:
    binned = pd.cut(df[col].astype(float),
bins=20)
    print(f"{col:20s} :
{discrete_entropy(binned):.3f}")
print(f"isDestMerchant :
{discrete_entropy(df['isDestMerchant']):.3f}")
print(f"type :
{discrete_entropy(df['type']):.3f}")
```

Kolom	Entropy	Interpretasi singkat
amount	0.017	Sangat terpusat/skewed - mayoritas nominal transaksi berada di rentang sempit
oldbalanceOrig	0.419	Variasi sedang
newbalanceOrig	0.475	Variasi sedang
oldbalanceDest	0.033	Sangat terpusat (didominasi banyak akun bersaldo 0, lih. temuan #6)
newbalanceDest	0.038	Sangat terpusat, alasan sama seperti di atas
isFraud	0.010	Sangat tidak seimbang (fraud sangat langka, sesuai proporsi ~0,13%)
step	2.486	Entropy tertinggi - waktu transaksi tersebar merata sepanjang periode
isDestMerchant	0.640	Variasi cukup, konsisten dengan proporsi ~34% transaksi ke merchant
type	1.308	Variasi tinggi - lima kategori transaksi dengan proporsi berbeda-beda

Entropy amount, oldbalanceDest, dan newbalanceDest yang sangat rendah konsisten dengan temuan sebelumnya (banyak akun merchant bersaldo tetap 0, dan distribusi amount yang sangat skewed) — ini menjadi salah satu alasan digunakannya RobustScaler pada tahap transformasi, bukan StandardScaler biasa.

1.6 Data Transformation

```
# One-hot encoding kolom type
type_dummies = pd.get_dummies(df["type"], prefix="type", dtype=np.int8)
df = pd.concat([df, type_dummies], axis=1)
df.drop(columns=["type"], inplace=True)
```

```
# RobustScaler untuk fitur numerik yang skewed/banyak outlier ekstrem
from sklearn.preprocessing import RobustScaler
from joblib import dump
scale_cols = ["amount", "oldbalanceOrg", "oldbalanceDest", "origError", "destError"]
scaler = RobustScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])
dump(scaler, '../models/robust_scaler.pkl')
```

RobustScaler dipilih (bukan StandardScaler) karena menggunakan median dan IQR, bukan mean dan standar deviasi — sehingga tidak mudah terdistorsi oleh outlier ekstrem pada amount dan saldo (konsisten dengan entropy amount yang sangat rendah/skewed pada bagian 1.5). Kolom step sengaja tidak ikut di-scale di sini, dan pada Phase 2 kolom ini juga dikeluarkan dari perhitungan jarak clustering karena sifatnya adalah indeks waktu, bukan atribut perilaku transaksi.

1.7 Feature Selection (Korelasi)

```
numeric_df = df.select_dtypes(include=[np.number])
corr_matrix = numeric_df.corr()
sns.heatmap(corr_matrix, annot=True, fmt=".2f")

cols_to_drop = ["newbalanceOrig", "newbalanceDest"]
df.drop(columns=cols_to_drop, inplace=True)
```

Correlation heatmap menunjukkan newbalanceOrig berkorelasi sangat kuat dengan oldbalanceOrg dan amount (karena secara matematis $\text{newbalanceOrig} \approx \text{oldbalanceOrg} - \text{amount}$ pada transaksi normal), begitu pula newbalanceDest dengan oldbalanceDest dan amount. Karena informasi keduanya sudah terwakili oleh origError/destError (yang secara eksplisit mengukur penyimpangan dari hubungan matematis tersebut) dan oldbalanceOrg/oldbalanceDest, kedua kolom ini dibuang untuk menghindari multikolinearitas berlebih pada tahap clustering di Phase 2.

1.8 Kesimpulan Phase 1

- Dataset PaySim bersih secara struktural: tidak ada nilai kosong, duplikat, ID collision, atau nilai negatif — sebagian besar "anomali" yang teridentifikasi (saldo merchant 0, ketidakcocokan saldo massal) adalah karakteristik desain simulator, bukan data rusak.
- Fraud di dataset ini sangat langka (8.197 dari 6.362.604 transaksi, $\approx 0,13\%$) dan hanya terjadi pada TRANSFER dan CASH_OUT, tidak pernah melibatkan merchant sebagai tujuan.
- Dua fitur turunan — origError/destError (ketidakesesuaian saldo) dan origDrainedToZero (akun dikuras habis) — terbukti punya keterkaitan kuat dengan fraud sejak tahap EDA, dan keduanya menjadi fondasi penting bagi Phase 2, 3, dan 4.
- Output akhir Phase 1: dataset bersih berjumlah 6.362.604 baris, siap dipakai untuk clustering (Phase 2).

Phase 2 — Clustering Analysis

Penanggung jawab: Segmentation Specialist. Input: 6.362.604 baris hasil Phase 1. Empat pendekatan clustering diterapkan pada data yang sama untuk tujuan yang berbeda: K-Means untuk segmentasi transaksi, HDBSCAN langsung dan kombinasi BIRCH + HDBSCAN untuk deteksi outlier, dan BIRCH untuk mendukung skema hybrid tersebut.

2.1 Persiapan: Fitur dan Reduksi Dimensi

```
df_clean = pd.read_parquet('../datasets/phase_1/paysim-dataset.parquet')
# 'step' dikeluarkan: ia adalah indeks waktu (1-743), bukan atribut perilaku transaksi,
# dan nilainya yang unscaled bisa mendominasi perhitungan jarak Euclidean.
X_clean = df_clean.drop(columns=['isFraud', 'step'], errors='ignore')

from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X_clean)
print(pca.explained_variance_ratio_)
print(pca.explained_variance_ratio_.sum())
```

Output

```
PC1 explained variance : 85.10%
PC2 explained variance : 11.63%
Total                  : 96.73%
```

PCA sempat dicoba dengan `n_components=0.95` (mempertahankan 95% variansi), namun jumlah komponen yang dihasilkan masih relatif banyak sehingga kurang efisien untuk clustering pada 6,3 juta baris. Karena dua komponen utama saja sudah mempertahankan 96,73% variansi total, PCA akhirnya difiksasi ke `n_components=2` — sekaligus memudahkan visualisasi dua dimensi. PCA 2 dimensi ini dipakai khusus untuk HDBSCAN langsung dan visualisasi; K-Means dan BIRCH tetap dilatih pada ruang fitur penuh (`X_clean`) agar tidak kehilangan informasi.

2.2 Penentuan Jumlah Cluster (K) untuk K-Means

```
from sklearn.metrics import davies_bouldin_score, calinski_harabasz_score
wcss, silhouette_scores, db_scores, ch_scores = [], [], [], []
for i in range(2, 8):
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_clean)
    wcss.append(kmeans.inertia_)
    sil_score = approximate_silhouette_score(X_clean, labels, kmeans.cluster_centers_)
    silhouette_scores.append(sil_score)
    db_scores.append(davies_bouldin_score(X_clean, labels))
    ch_scores.append(calinski_harabasz_score(X_clean, labels))
```

Catatan metodologi: karena silhouette score standar terlalu berat secara komputasi untuk 6,3 juta baris, digunakan pendekatan approximate silhouette score (jarak titik ke centroid sendiri vs. centroid terdekat lain), bukan implementasi sklearn yang mengevaluasi seluruh pasangan titik.

K	WCSS (inertia)	Silhouette (approx.)	Davies-Bouldin (DBI)	Calinski-Harabasz
2	2.093.408.996,89	0,9152	0,4700	10.160.519,70
3	1.275.538.364,67	0,8973	0,4851	10.377.541,66
4	921.160.697,54	0,8978	0,4591	10.395.819,18
5	709.364.556,38	0,8674	0,5338	10.599.710,40

K	WCSS (inertia)	Silhouette (approx.)	Davies-Bouldin (DBI)	Calinski-Harabasz
6	610.136.916,18	0,8510	0,5654	10.065.807,27
7	523.139.211,71	0,8518	0,5691	9.959.466,35

Catatan objektif: silhouette score aproksimasi sebenarnya tertinggi di K=2 (0,9152), bukan di K=4 seperti yang sempat dituliskan di narasi awal notebook (yang menyebut nilai 0,6869 - angka ini tidak konsisten dengan output aktual dan sebaiknya dikoreksi pada laporan akhir). Namun demikian, Davies-Bouldin Index (semakin rendah semakin baik) justru minimum secara jelas di K=4 (0,4591), dan WCSS pada grafik elbow menunjukkan penurunan yang mulai melandai setelah K=4. K=2 memisahkan data terlalu kasar untuk kebutuhan segmentasi bisnis (hanya 2 kelompok), sedangkan K=4 memberi keseimbangan terbaik antara kekompakan cluster (DBI terendah) dan jumlah segmen yang masih bisa diinterpretasikan secara bisnis. Atas dasar itu, K=4 dipilih untuk K-Means.

2.3 K-Means Clustering

```
kmeans = KMeans(n_clusters=4, init='k-means++', random_state=42, n_init=10)
df_clean['cluster_kmeans'] =
kmeans.fit_predict(X_clean)
```

Cluster	Jumlah transaksi	% dari total	Fraud rate	Karakteristik dominan
0	1.648	0,03%	0,00%	TRANSFER (79%) & CASH_OUT (13%); origError dan destError jauh di atas rata-rata
1	5.880.042	92,42%	0,12%	Campuran semua tipe transaksi (CASH_OUT 38%, PAYMENT 37%, CASH_IN 16%, TRANSFER 9%) - aktivitas perbankan umum
2	391.249	6,15%	0,26%	100% CASH_IN, oldbalanceOrg relatif tinggi
3	89.665	1,41%	0,11%	100% CASH_IN, oldbalanceOrg jauh lebih tinggi dari cluster 2

Temuan: K-Means menghasilkan empat cluster dengan distribusi yang jelas berjenjang tapi tetap saling terkait dengan

pola bisnis: satu cluster raksasa berisi aktivitas perbankan umum (92,4%), dua cluster berbasis CASH_IN dengan level saldo berbeda, dan satu cluster sangat kecil (0,03%) dengan aktivitas TRANSFER/CASH_OUT dan penyimpangan saldo (origError/destError) yang jauh di atas rata-rata cluster lain. Fraud rate antar-cluster relatif berdekatan (0-0,26%), menunjukkan K-Means membagi data berdasarkan pola perilaku transaksi, bukan berdasarkan status fraud.

2.4 HDBSCAN Langsung (Tanpa BIRCH)

```
import hdbscan
# Stratified sampling 300.000 baris (proporsional per tipe transaksi) untuk fit
HDBSCAN,
# karena fit langsung pada 6,3 juta baris tidak feasible secara komputasi
hdbscan_cpu = hdbscan.HDBSCAN(
    min_cluster_size=3000, min_samples=20,
    prediction_data=True, core_dist_n_jobs=-1, cluster_selection_method='eom'
)
hdbscan_cpu.fit(X_reduced_sample)

# approximate_predict dijalankan per-batch (500.000 baris) untuk seluruh 6,3 juta data
batch_size = 500000
all_labels = []
for i in range(0, len(X_reduced), batch_size):
    batch_labels, _ = hdbscan.approximate_predict(hdbscan_cpu,
X_reduced[i:i+batch_size])
    all_labels.extend(batch_labels)
df_clean['cluster_hdbscan'] = all_labels
```

Output

```
Outlier count (label -1) : 4.526.339 (~71,17% dari seluruh data)
Jumlah cluster valid    : 7
Cluster terbesar       : 929.513 baris
```

Cluster terkecil : 95.458 baris

Temuan: HDBSCAN yang dijalankan langsung menandai lebih dari 70% transaksi (4.526.339 dari 6.362.604) sebagai outlier (label -1) — jauh melampaui proporsi yang wajar untuk deteksi anomali. Ini menunjukkan bahwa pada ruang fitur PCA 2 dimensi, data transaksi finansial tidak membentuk wilayah kepadatan (density region) yang cukup jelas terpisah sesuai asumsi dasar HDBSCAN, sehingga metode ini dinilai tidak cocok dipakai sendirian pada dataset ini.

2.5 BIRCH

Untuk mengatasi keterbatasan HDBSCAN pada skala 6,3 juta baris, BIRCH digunakan sebagai langkah kompresi awal karena efisien membangun CF Tree tanpa harus memuat seluruh data ke memori sekaligus.

```
# Uji beberapa nilai
threshold pada sampel
data untuk memilih
konfigurasi BIRCH
thresholds = [2, 4, 6,
8, 12, 16, 20, 24, 32]
for t in thresholds:
    birch =
    Birch(threshold=t,
branching_factor=50,
n_clusters=None)

    birch.fit(X_birch_tune)
    # ... catat jumlah
    subcluster & waktu
    eksekusi
```

Threshold	Jumlah subcluster	Waktu (detik)
2	1.903	4,06
4	667	2,78
6	316	1,59
8	210	1,54
12	110	1,01
16	77	1,35
20	49	0,77
24	43	0,73
32	35	0,75

Threshold = 4 dipilih sebagai keseimbangan antara mempertahankan struktur data (subcluster lebih banyak/detail) dan efisiensi komputasi.

```
birch = Birch(threshold=4, branching_factor=50,
n_clusters=None)
birch.fit(X_clean) # dilatih pada seluruh
6.362.604 baris
print('Subclusters:',
len(birch.subcluster_centers_))
```

```
# MiniBatchKMeans mengelompokkan subcluster
menjadi 4 cluster global (agar sebanding dengan
K-Means)
mbk = MiniBatchKMeans(n_clusters=4,
random_state=42, batch_size=512)
mbk.fit(subcluster_centers)
subcluster_labels = birch.predict(X_clean)
df_clean['cluster_birch'] =
mbk.labels_[subcluster_labels]
```

Output

Subclusters: 5.341

Cluster	Jumlah transaksi	% dari total	Fraud rate
0	2.703	0,04%	0,04%
1	434	0,01%	0,00%
2	6.357.197	99,92%	0,13%
3	2.270	0,04%	10,35%

Temuan: BIRCH berhasil membentuk 5.341 subcluster yang merepresentasikan seluruh dataset secara efisien, namun ketika subcluster-subcluster ini dikelompokkan menjadi 4 cluster global, hasilnya sangat timpang: 99,92% data jatuh ke satu cluster dominan (cluster 2). Meski demikian, satu cluster sangat kecil (cluster 3, hanya 2.270 baris) berhasil menangkap kelompok dengan fraud rate 10,35% — jauh di atas

rata-rata keseluruhan (0,13%). Ini menunjukkan BIRCH sendiri kurang cocok untuk segmentasi umum (karena satu cluster mendominasi hampir seluruh data), tetapi cukup baik dalam mengisolasi kelompok kecil yang menyimpang.

2.6 BIRCH + HDBSCAN (Hybrid) — Metode Akhir untuk Deteksi Outlier

Untuk mengatasi kelemahan HDBSCAN langsung (Bagian 2.4), HDBSCAN diterapkan bukan pada 6,3 juta transaksi individual, melainkan pada 5.341 titik pusat subcluster BIRCH — sehingga jauh lebih ringan secara komputasi namun tetap mempertahankan struktur data yang direpresentasikan BIRCH.

```
subcluster_centers =
birch.subcluster_centers_.astype(np.float32)
hdbscan_birch = hdbscan.HDBSCAN(
    min_cluster_size=30, min_samples=10,
    cluster_selection_method='eom',
    prediction_data=True
)
cluster_labels =
hdbscan_birch.fit_predict(subcluster_centers)

# Setiap transaksi mewarisi label HDBSCAN dari
subcluster BIRCH tempat ia berada
subcluster_labels = birch.predict(X_clean)
df_clean['cluster_birch_hdbscan'] =
cluster_labels[subcluster_labels]
```

Cluster	Jumlah transaksi	% dari total	Fraud rate
-1 (outlier)	3.923	0,06%	16,93%
0	156	0,00%	0,00%
1	292	0,00%	0,00%
2	886	0,01%	0,00%
3 (dominan)	6.357.347	99,93%	0,12%

Temuan:
Dibanding

HDBSCAN langsung yang menandai 71,17% data sebagai outlier, kombinasi BIRCH + HDBSCAN hanya menandai 3.923 transaksi (0,06%) sebagai outlier — namun kelompok kecil ini memiliki fraud rate 16,93%, sekitar 131 kali lipat dari fraud rate keseluruhan dataset (0,13%). Ini adalah bukti kuat bahwa pendekatan hybrid berhasil memusatkan transaksi yang benar-benar mencurigakan ke dalam kelompok yang jauh lebih kecil dan presisi, dan menjadikannya metode akhir untuk deteksi outlier pada Phase 2, yang kemudian dipakai

sebagai input
Phase 3 dan
Phase 4.

2.7 Perbandingan Keempat Metode dan Alasan Pemilihan K-Means sebagai Acuan Segmentasi

Metode	Cakupan cluster valid	Proporsi "outlier"/noise	Cocok untuk
K-Means (k=4)	100% data terbagi ke 4 cluster	Tidak ada konsep outlier	Segmentasi transaksi (seluruh data terwakili, cluster seimbang & mudah diinterpretasikan)
HDBSCAN langsung	7 cluster + noise	71,17%	Tidak layak dipakai - noise terlalu dominan
BIRCH (+ MiniBatchKMeans)	4 cluster (99,92% di 1 cluster)	Tidak ada konsep outlier, tapi timpang	Kompresi data (CF Tree), bukan segmentasi akhir
BIRCH + HDBSCAN	4 cluster + outlier	0,06%	Deteksi outlier presisi tinggi (fraud rate 131x baseline)

K-Means (K=4) dipilih sebagai acuan utama untuk customer/transaction segmentation, dengan tiga alasan utama:

- Cakupan penuh dan seimbang: seluruh 6.362.604 transaksi terbagi habis ke 4 cluster tanpa konsep "noise"/outlier, sehingga setiap transaksi bisa diberi label segmen yang jelas - cocok untuk kebutuhan segmentasi bisnis yang butuh setiap nasabah/transaksi masuk ke satu kelompok.
- Validitas terukur: Davies-Bouldin Index minimum secara jelas di K=4 dibanding K lain yang diuji (lihat Bagian 2.2), menunjukkan cluster yang dihasilkan cukup kompak dan terpisah.
- Interpretasi bisnis yang jelas: setiap cluster K-Means punya karakteristik dominan yang mudah diberi nama bisnis (lihat Bagian 2.8), berbeda dengan BIRCH yang 99,92% datanya terjebak di satu cluster tunggal (sulit dibedakan secara bisnis), atau HDBSCAN yang 71% datanya berstatus noise (tidak punya label sama sekali).

Sebaliknya, kombinasi BIRCH + HDBSCAN sengaja tidak dipakai untuk segmentasi umum karena tujuannya memang berbeda: mengisolasi sekelompok sangat kecil transaksi yang menyimpang jauh dari populasi normal (deteksi outlier/anomali), bukan membagi seluruh populasi menjadi kelompok-kelompok perilaku yang seimbang. Kedua metode ini saling melengkapi, bukan saling menggantikan: K-Means menjawab "transaksi ini termasuk kelompok perilaku yang mana", sedangkan BIRCH+HDBSCAN menjawab "transaksi ini menyimpang secara ekstrem atau tidak".

2.8 Customer / Transaction Segmentation (berdasarkan K-Means)

Berdasarkan profil rata-rata setiap cluster K-Means (Bagian 2.3), keempat cluster dapat diberi nama dan interpretasi bisnis sebagai berikut:

Segmen 1 — "Aktivitas Perbankan Umum" (Cluster 1, 92,42% data)

Segmen terbesar dan paling beragam, mencakup kombinasi CASH_OUT, PAYMENT, CASH_IN, dan TRANSFER. Fraud rate 0,12%, mendekati rata-rata dataset. Ini merepresentasikan mayoritas nasabah dengan pola transaksi sehari-hari yang wajar.

Segmen 2 — "Setoran Tunai / Cash-In Reguler" (Cluster 2, 6,15% data)

100% transaksi CASH_IN dengan saldo awal (oldbalanceOrg) relatif tinggi. Fraud rate 0,26% — tertinggi di antara keempat segmen K-Means, meski secara absolut tetap rendah. Menarik untuk dipantau meski tidak dominan.

Segmen 3 — "Nasabah Bersaldo Tinggi (Cash-In)" (Cluster 3, 1,41% data)

Mirip Segmen 2 (100% CASH_IN) namun dengan oldbalanceOrg jauh lebih tinggi lagi — kemungkinan mewakili nasabah/agen dengan kapasitas dana besar. Fraud rate 0,11%, sedikit di bawah rata-rata.

Segmen 4 — "Transfer Bernilai Tinggi dengan Anomali Saldo" (Cluster 0, 0,03% data)

Segmen terkecil namun paling khas: didominasi TRANSFER (79%) dan CASH_OUT (13%), dengan origError dan destError jauh di atas rata-rata segmen lain — menandakan penyimpangan saldo yang signifikan. Fraud rate tercatat 0,00% pada level agregat K-Means, namun segmen ini beririsan kuat dengan kelompok outlier BIRCH+HDBSCAN (Bagian 2.6) yang justru punya fraud rate 16,93%. Artinya, segmen ini bukan berarti "aman", melainkan populasi kecil dengan karakteristik tidak biasa yang layak dipantau lebih lanjut - konsisten dengan rekomendasi Phase 4.

2.9 Kesimpulan Phase 2

- Empat algoritma clustering diterapkan dengan tujuan berbeda: K-Means untuk segmentasi bisnis (acuan utama), HDBSCAN langsung sebagai baseline yang terbukti tidak layak (71% noise), BIRCH sebagai kompresi data, dan BIRCH+HDBSCAN sebagai metode akhir deteksi outlier.
- K-Means (K=4) dipilih sebagai acuan segmentasi karena mencakup 100% data, tervalidasi Davies-Bouldin Index, dan menghasilkan 4 segmen yang mudah diinterpretasikan secara bisnis.
- BIRCH + HDBSCAN berhasil menekan proporsi outlier dari 71,17% (HDBSCAN langsung) menjadi 0,06%, sambil mengonsentrasikan fraud rate dari 0,13% (baseline) menjadi 16,93% pada kelompok outlier tersebut - hasil ini menjadi input utama bagi Phase 3 dan Phase 4.
- Kolom cluster_kmeans dan cluster_birch_hdbscan (bukan cluster_hdbscan langsung) adalah dua output Phase 2 yang dipakai secara konsisten pada Phase 3 dan Phase 4.

Phase 3 — Association Rule Mining

Penanggung jawab: Pattern Analyst. Input: 6.362.604 baris hasil Phase 2 (lengkap dengan cluster_kmeans dan cluster_birch_hdbscan). Tujuan: menemukan pola ko-kejadian (co-occurrence) antar-atribut yang tidak terlihat lewat tabulasi sederhana.

3.1 Konstruksi Basket (Transaksi sebagai Kumpulan Item)

Karena Apriori bekerja pada data transaksi biner (item ada/tidak ada), setiap kolom numerik/kategorikal diubah menjadi item biner:

```
def build_basket(df):
    basket_parts = []
    # Tipe transaksi (one-hot dari Phase 1)
    type_cols = [c for c in df.columns if c.startswith('type_')]
    basket_parts.append(df[type_cols].astype(bool))

    # Label fraud, merchant, dan akun terkuras (biner ya/tidak)
    basket_parts.append(pd.DataFrame({
        'isFraud_yes': df['isFraud'] == 1, 'isFraud_no': df['isFraud'] == 0}))
    # ... isDestMerchant dan origDrainedToZero dengan pola serupa

    # Cluster K-Means (Phase 2) di-one-hot
    basket_parts.append(pd.get_dummies(df['cluster_kmeans'].astype(str),
    prefix='cluster_kmeans'))

    # Outlier BIRCH+HDBSCAN (Phase 2) - BUKAN cluster_hdbscan langsung
    basket_parts.append(pd.DataFrame({
        'hdbscan_outlier': df['cluster_birch_hdbscan'] == -1,
        'hdbscan_normal': df['cluster_birch_hdbscan'] != -1}))

    # Diskretisasi 5 fitur numerik (amount, oldbalanceOrg, oldbalanceDest, origError,
    destError)
    # menjadi 5 bin kuantil: very_low, low, medium, high, very_high
    for col in ['amount', 'oldbalanceOrg', 'oldbalanceDest', 'origError', 'destError']:
        cat = discretize_numeric_series(df[col], col_name=col, q=5)
        basket_parts.append(pd.get_dummies(cat))
    return pd.concat(basket_parts, axis=1).fillna(False)
```

Catatan penting: item hdbscan_outlier di-derive dari cluster_birch_hdbscan (metode hybrid Phase 2), bukan dari cluster_hdbscan hasil HDBSCAN langsung — karena HDBSCAN langsung menandai ~71% baris sebagai outlier (lihat Phase 2, Bagian 2.4), yang membuat item ini nyaris tidak informatif untuk association rule mining.

3.2 Apriori dan Pembentukan Rule

```
from mlxtend.frequent_patterns import apriori, association_rules
MIN_SUPPORT, MIN_CONFIDENCE, MIN_LIFT, MAX_LEN = 0.005, 0.50, 1.20, 3

frequent_itemsets = apriori(basket, min_support=MIN_SUPPORT, use_colnames=True,
max_len=MAX_LEN)
rules = association_rules(frequent_itemsets, metric='lift', min_threshold=MIN_LIFT)
rules = rules[(rules['confidence'] >= MIN_CONFIDENCE) & (rules['lift'] >= MIN_LIFT)]

# Apriori terpisah dengan min_support jauh lebih rendah (0.0001) KHUSUS untuk pola
fraud,
# karena fraud adalah kelas yang sangat langka (~0,13%) sehingga akan lolos begitu saja
```

```
# oleh MIN_SUPPORT=0.005 di atas jika dipaksakan pakai satu threshold yang sama
fraud_itemsets = apriori(basket, min_support=0.0001, use_colnames=True, max_len=3)
fraud_rules = association_rules(fraud_itemsets, metric='lift', min_threshold=1.2)
fraud_rules = fraud_rules[fraud_rules['consequents_str'].str.contains('isFraud_yes')]
```

Rule kemudian disaring lebih lanjut (`filter_meaningful_rules` dan `generate_report_rules`) untuk membuang consequent yang terlalu umum/trivial (`isFraud_no`, `hdbscan_normal`, `orig_drained_no`, `dest_merchant_no`), sehingga yang tersisa adalah rule yang benar-benar mengarah ke pola yang actionable secara bisnis.

3.3 Sepuluh Rule Teratas (Berdasarkan Lift)

#	Jika (antecedent)	Maka (consequent)	Support	Confidence	Lift
1	origError sangat rendah + akun asal dikuras habis	Fraud	0,13%	100,0%	776,2
2	Cluster K-Means 2 + akun asal dikuras habis	Fraud	0,02%	100,0%	776,2
3	Outlier BIRCH+HDBSCAN + origError sangat rendah	Fraud	0,01%	90,4%	702,0
4	Cluster K-Means 2 + origError sangat rendah	Fraud	0,02%	65,0%	504,3
5	Tipe transaksi DEBIT	amount sangat rendah + Cluster K-Means 1	0,59%	90,7%	4,6
6	destError rendah	amount sangat rendah + Cluster K-Means 1	17,64%	88,2%	4,5
7	Tipe transaksi DEBIT	amount sangat rendah + destError sangat rendah	0,54%	83,2%	47,2
8	Cluster K-Means 3	oldbalanceOrg sangat tinggi + CASH_IN	1,41%	99,8%	7,3
9	Cluster K-Means 2	oldbalanceOrg sangat tinggi + CASH_IN	6,12%	99,6%	7,3
10	Cluster K-Means 2	destError sangat tinggi + oldbalanceOrg sangat tinggi	4,88%	79,3%	7,2

Lift dihitung sebagai $P(\text{consequent} | \text{antecedent}) / P(\text{consequent})$ - lift = 776 berarti kombinasi tersebut 776 kali lebih sering diikuti fraud dibanding jika antecedent dan consequent independen.

3.4 Interpretasi Bisnis

Temuan: Rule #1 dan #2 (lift 776,2, confidence 100%): setiap kali sebuah transaksi memiliki penyimpangan saldo asal yang nyaris nol (origError sangat rendah - artinya saldo tercatat konsisten secara matematis) DAN akun pengirimnya dikuras habis, transaksi tersebut SELALU berstatus fraud dalam data ini. Temuan ini menarik karena berlawanan dengan intuisi awam bahwa "kesalahan pembukuan" adalah tanda fraud; justru pola pengurusan akun yang rapi secara aritmatika (bukan berantakan) yang menjadi sinyal fraud paling kuat.

Temuan: Rule #3 (lift 702,0, confidence 90,4%): transaksi yang sudah ditandai sebagai outlier oleh model BIRCH+HDBSCAN Phase 2, dan sekaligus punya origError sangat rendah, berstatus fraud pada 90,4% kasus. Ini adalah validasi silang yang kuat antara Phase 2 (clustering) dan Phase 3 (association rule) - keduanya independen tapi mengarah ke kesimpulan yang konsisten.

Temuan: Rule #5–7: sebagian besar transaksi bertipe DEBIT, atau dengan destError rendah, cenderung berada di Cluster K-Means 1 (segmen "Aktivitas Perbankan Umum") dengan nominal transaksi kecil. Rule ini lebih bersifat deskriptif/konfirmatif terhadap definisi segmen Phase 2, bukan sinyal fraud, namun membuktikan bahwa segmen K-Means konsisten dengan pola transaksi riil.

Temuan: Rule #8–10: Cluster K-Means 2 dan 3 (segmen berbasis CASH_IN, lihat Phase 2 Bagian 2.8) hampir selalu diikuti oleh oldbalanceOrg sangat tinggi dan tipe transaksi CASH_IN (confidence 99,6-99,8%) - rule ini menegaskan validitas definisi kedua segmen tersebut. Rule #10 tambahan menunjukkan Cluster 2 juga cenderung punya destError sangat tinggi, sinyal tambahan yang sejalan dengan catatan di Phase 2 bahwa segmen ini perlu tetap dipantau meski fraud rate absolutnya rendah.

3.5 Kesimpulan Phase 3

- Pola "akun dikuras habis + saldo konsisten secara matematis" adalah sinyal fraud terkuat yang ditemukan di seluruh proyek ini (lift 776, confidence 100%), dan bisa langsung dijadikan aturan bisnis (business rule) sederhana untuk sistem peringatan dini.
- Association rule mining memberikan validasi silang independen terhadap hasil clustering Phase 2 - baik untuk kelompok outlier (BIRCH+HDBSCAN) maupun segmen K-Means - keduanya saling menguatkan, bukan bertentangan.
- Sepuluh rule di atas seluruhnya non-trivial (bukan sekadar duplikasi definisi tipe transaksi) dan memenuhi syarat rubrik: didukung support, confidence, dan lift yang jelas serta interpretasi bisnis yang spesifik.

Phase 4 — Anomaly Detection

Penanggung jawab: Data Engineer & Pattern Analyst. Input: 6.362.604 baris hasil Phase 2, ditambah 3.923 baris outlier BIRCH+HDBSCAN (paysim-outliers-phase4.parquet). Bagian ini merefleksikan versi phase_4 terbaru, yang memperluas versi sebelumnya dengan pembobotan sinyal, flag ketidaksesuaian saldo tambahan, dan kategorisasi investigasi otomatis.

```
df, df_outliers = load_data(path_1, path_2)
# Sanity check konsistensi Phase 2 -> Phase 4
expected_outlier_count = int((df['cluster_birch_hdbscan'] == -1).sum())
```

Output

```
Data shape          : (6362604, 18)
Data outliers shape : (3923, 18)
Rows cluster_birch_hdbscan == -1 : 3.923
Rows in outlier parquet      : 3.923
Status                      : KONSISTEN
```

4.1 Empat Metode Deteksi Anomali

a) IQR (univariat, kolom amount)

```
Q1, Q3 = df['amount'].quantile([0.25, 0.75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
flag_IQR = ((df['amount'] < lower) | (df['amount'] > upper)).astype(int)
```

Output

```
Q1=-0,3148 | Q3=0,6852 | IQR=1,0000
Lower=-1,8148 | Upper=2,1852
Total anomali: 338.077 (5,31%)
Nilai maks anomali: 472,89 | Nilai min anomali: 2,19
```

b) Z-Score (univariat, kolom amount, ambang $|Z| > 3$)

```
mean, std = df['amount'].mean(), df['amount'].std()
z = (df['amount'] - mean) / std
flag_ZScore = (z.abs() > 3).astype(int)
```

Output

```
Mean=0,5375 | Std=3,0914 | Threshold  $|Z|>3$ 
Total anomali: 44.945 (0,71%)
Nilai maks anomali: 472,89 | Nilai min anomali: 9,81
```

c) Isolation Forest (multivariat: amount, origError, destError)

```
feature_cols = ['amount', 'origError', 'destError']
model = IsolationForest(contamination=0.01, random_state=42)
preds = model.fit_predict(df[feature_cols].fillna(0))
flag_IsoForest = (preds == -1).astype(int)
```

Output

```
Fitur: amount, origError, destError | Contamination: 0,01
Total anomali: 63.616 (1,00%)
Maks origError anomali: 374,47 | Maks destError anomali: 1.747,51
```

d) Balance Mismatch (tambahan baru: IQR pada origError & destError)

Metode baru yang ditambahkan pada revisi ini, khusus untuk menandai ketidakwajaran saldo (bukan nominal transaksi) sebagai bahan investigasi terpisah dari deteksi fraud murni:

```
def detect_balance_mismatch(df, columns=('origError', 'destError')):
    flags = []
    for col in columns:
        Q1, Q3 = df[col].quantile([0.25, 0.75])
        IQR = Q3 - Q1
        lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
        flags.append(((df[col] < lower) | (df[col] > upper)).astype(int))
    return pd.concat(flags, axis=1).max(axis=1).astype(int)
```

Output

```
Total balance mismatch: 1.483.549 (23,32%)
origError: lower=-1,7664 upper=2,2336, flagged=408.487
destError: lower=-1,6180 upper=2,3820, flagged=1.286.173
```

Kelima flag dari BIRCH+HDBSCAN (Phase 2) dan keempat metode di atas kemudian digabung menjadi flag_IQR, flag_ZScore, flag_IsoForest, flag_HDBSCAN, dan flag_BalanceMismatch.

4.2 Skor Risiko Berbobot dan Kategorisasi

Berbeda dari versi sebelumnya yang menjumlahkan flag secara setara, revisi ini memberi bobot lebih besar pada metode yang menangkap pola multivariat/cluster (Isolation Forest dan BIRCH+HDBSCAN), karena keduanya mempertimbangkan lebih dari satu dimensi sekaligus, sementara IQR dan Z-Score hanya membaca kolom amount:

```
SIGNAL_WEIGHTS = {'flag_IQR': 1,
                  'flag_ZScore': 1, 'flag_IsoForest':
                  2, 'flag_HDBSCAN': 2}
HIGH_RISK_THRESHOLD = 3      #
    butuh minimal 1 sinyal kuat + 1
    sinyal lain
CRITICAL_RISK_THRESHOLD = 5  #
    butuh IsoForest DAN HDBSCAN, plus
    minimal 1 sinyal lain

df['risk_score'] = (df['flag_IQR']*1
+ df['flag_ZScore']*1
+
df['flag_IsoForest']*2 +
df['flag_HDBSCAN']*2)
df['high_risk'] = (df['risk_score']
>= HIGH_RISK_THRESHOLD).astype(int)
df['critical_risk'] =
(df['risk_score'] >=
CRITICAL_RISK_THRESHOLD).astype(int)
df['risk_level'] = np.select(
    [df.risk_score>=5,
df.risk_score>=3, df.risk_score==2,
df.risk_score==1],
    ['Critical', 'High', 'Medium',
'Low'], default='Normal')
```

Risk level	Jumlah transaksi	% dari total
Normal (score 0)	6.023.646	94,67%
Low (score 1)	273.596	4,30%
Medium (score 2)	1.645	0,026%

High (score 3-4)	60.776	0,96%
Critical (score >=5)	2.941	0,046%

4.3 Klasifikasi Jenis Anomali dan Kategori Investigasi

Selain skor numerik, setiap baris juga diberi label `anomaly_type` (jenis anomali apa yang terdeteksi) dan `investigation_category` (data error / rare legitimate case / potential risk, mengikuti kerangka Phase 4 pada dokumen proyek):

```
df['investigation_category'] = np.select(
    [
        ((df.isFraud==1) & (df.high_risk==1)) |
        (df.risk_score>=CRITICAL_RISK_THRESHOLD),
        (df.flag_BalanceMismatch==1) & (df.isFraud==0),
        (df.isFraud==0) & (df.high_risk==1),
        df.risk_indicator_count>=1,
    ],
    ['Potential Fraud', 'Possible Data Quality Issue',
     'Rare Legitimate Transaction', 'Potential Risk / Monitor'],
    default='Normal / Low Concern')
```

Catatan metodologi: kategori Potential Fraud di sini memakai label `isFraud` yang sudah diketahui, sehingga sifatnya adalah validasi retrospektif (mengonfirmasi kasus yang berhasil ditangkap sistem), bukan prediksi murni tanpa label — sesuai konteks tugas ini yang memang meminta cross-reference dan investigasi terhadap hasil deteksi, bukan membangun model prediktif untuk data baru yang labelnya belum diketahui.

Jenis anomali (<code>anomaly_type</code>)	Jumlah	%
No Statistical Anomaly	4.793.091	75,33%
Balance Mismatch	1.439.370	22,62%
Extreme Transaction Amount	85.685	1,35%
Multiple Indicators	44.265	0,70%
Cluster Outlier (BIRCH+HDBSCAN)	185	0,003%
Behavioral and Cluster Outlier	8	0,0001%
Kategori investigasi	Jumlah	%
Normal / Low Concern	4.794.398	75,35%
Possible Data Quality Issue	1.477.915	23,23%
Potential Risk / Monitor	86.508	1,36%
Potential Fraud	3.780	0,059%
Rare Legitimate Transaction	3	0,00005%

Temuan:
Kategori
"Possible Data
Quality Issue"

Jenis anomali (anomaly_type)	Jumlah	%
mendominasi (23,23% dari seluruh data) - ini menunjukkan bahwa mayoritas penyimpangan yang terdeteksi bukan fraud, melainkan ketidaksesuaian saldo yang sifatnya sistemik (konsisten dengan Phase 1 temuan #7 dan #9), sehingga penting untuk tidak menyamarkan "anomali" dengan "fraud" pada laporan ke pemangku kepentingan bisnis.		

4.4 Validasi terhadap Label Fraud Aktual

```

y_true, y_pred = df['isFraud'], df['high_risk']
TP = ((y_true==1)&(y_pred==1)).sum(); FP =
((y_true==0)&(y_pred==1)).sum()
FN = ((y_true==1)&(y_pred==0)).sum(); TN =
((y_true==0)&(y_pred==0)).sum()
precision, recall = TP/(TP+FP), TP/(TP+FN)
f1 = 2*precision*recall/(precision+recall)

```

Metrik	Nilai
High risk transactions	63.717 (1,00% dari total)
True Positive (TP)	1.429
False Positive (FP)	62.288
False Negative (FN)	6.768
True Negative (TN)	6.292.119
Precision	2,24%
Recall	17,43%

F1-score	3,97%
Fraud rate keseluruhan (baseline)	0,1288%
Fraud rate dalam high_risk	2,24%
Enrichment (pengayaan)	17,4x baseline

Temuan:

Precision sistem berbobot ini tergolong rendah (2,24%) karena kelompok high_risk didominasi transaksi yang punya nominal besar tapi sah (whale transactions) - Isolation Forest dan IQR sama-sama mudah terpicu oleh nominal ekstrem yang belum tentu fraud. Namun demikian, recall 17,43% berarti sistem tetap berhasil menangkap sekitar 1 dari 6 kasus fraud yang ada, dan enrichment 17,4x menunjukkan kelompok high_risk

tetap jauh lebih terkonsentrasi fraud-nya dibanding populasi umum. Precision yang rendah ini adalah alasan penting untuk menggunakan pendekatan multi-tingkat (risk_level bertingkat, bukan sekadar high/low), sehingga penyaringan lebih lanjut (misalnya berbasis kategori Critical atau kombinasi dengan rule Phase 3) tetap diperlukan sebelum eskalasi ke tim investigasi.

4.5 Fraud Rate per Level Skor Risiko — Sebuah Nuansa Penting

```
fraud_by_score =
df.groupby('risk_score')['isFraud'].agg(['count', 'sum', 'mean'])
```

Risk score (berbobot)	Jumlah transaksi	Jumlah fraud	Fraud rate
0	6.023.646	4.342	0,072 %

1	273.596	1.697	0,620 %
2	1.645	729	44,32 %
3	19.444	471	2,42 %
4	41.332	368	0,89 %
5	92	0	0,00 %
6 (seluruh sinyal)	2.849	590	20,71 %

Temuan: Hasil ini menunjukkan pola yang TIDAK monoton: skor 2 justru punya fraud rate tertinggi (44,32%), melebihi skor 6/maksimum (20,71%) sekalipun. Ini terjadi karena skor 2 bisa dicapai lewat tiga kombinasi berbeda dengan bobot yang sama (IQR+ZScore bersamaan, ATAU Isolation Forest sendirian, ATAU BIRCH+HDBSCAN sendirian) - dan populasi kecil (1.645 baris) yang benar-benar

mencapai skor 2 di data ini didominasi oleh kombinasi yang justru sangat presisi (kemungkinan besar BIRCH+HDBSCAN atau Isolation Forest tanpa disertai sinyal amount yang umum). Temuan ini penting dicatat sebagai keterbatasan skema pembobotan linear: skor total yang sama tidak selalu berarti tingkat risiko yang setara, karena bergantung kombinasi metode mana yang menyusunnya, bukan sekadar jumlah/bobotnya. Ini adalah bahan investigasi lanjutan yang baik untuk laporan pengetahuan

(Phase 5) -
bahwa
kombinasi
sinyal mana
yang aktif,
sama
pentingnya
dengan
berapa banyak
sinyal yang
aktif.

4.6 Cross-Reference dengan Segmen K-Means (Phase 2)

```
cluster_summary =
df.groupby('cluster_kmeans').agg(
    transactions=('isFraud', 'size'),
    fraud_rate=('isFraud', 'mean'),
    avg_risk_score=('risk_score', 'mean'),
    high_risk_count=('high_risk', 'sum'))
```

Segmen K-Means (Phase 2)	Rata-rata risk_score	% high risk	Fraud rate aktual
Segmen 4 - Transfer Bernilai Tinggi (cluster 0)	4,66	79,13%	0,00%
Segmen 2 - Setoran Tunai Reguler (cluster 2)	0,04	0,71%	0,26%
Segmen 3 - Nasabah Bersaldo Tinggi (cluster 3)	0,05	1,00%	0,11%
Segmen 1 - Aktivitas Perbankan Umum (cluster 1)	0,08	1,00%	0,12%

Temuan: Temuan paling menarik pada bagian cross-reference ini: Segmen 4 (K-Means cluster 0, hanya 1.648 transaksi, lihat Phase 2 Bagian 2.8) memiliki rata-rata skor risiko 4,66 - puluhan kali lebih tinggi dari tiga segmen lain - dan 79,13% dari transaksinya masuk kategori high_risk. Namun

fraud rate
aktualnya tetap
0,00%. Ini adalah
contoh nyata
kategori "Rare
Legitimate
Transaction"
dalam kerangka
investigasi Phase
4: segmen ini
secara statistik
terlihat sangat
menyimpang
(nominal besar,
origError/destError
tinggi), tetapi
berdasarkan label
yang tersedia
bukan merupakan
fraud -
kemungkinan
besar
merepresentasikan
transaksi bernilai
besar yang sah
namun jarang
terjadi (misalnya
transfer
korporat/agen
bernilai besar).
Segmen ini tetap
layak dipantau
(karena ukurannya
sangat kecil dan
pola tidak lazim),
namun tidak
seharusnya
diperlakukan sama
dengan sinyal
fraud.

4.7 Transaksi Paling Mencurigakan (Top 20.000)

```
top_df = df.sort_values(
```



```
by=['risk_score','risk_indicator_count','flag_BalanceMismatch','amount'],
ascending=False).head(20000)
```

Output

```
Fraud rate dalam subset top-20.000: 2,99% (~23x baseline 0,13%)
Kategori investigasi: Possible Data Quality Issue 17.051 | Potential Fraud 2.949
Breakdown tipe transaksi: TRANSFER 19.931 | CASH_IN 49 | CASH_OUT 20
```

Hampir seluruh (99,7%) transaksi yang paling mencurigakan berjenis TRANSFER, konsisten dengan temuan Phase 1 (fraud hanya pada TRANSFER dan CASH_OUT) dan Phase 3 (rule fraud dengan lift tertinggi selalu melibatkan pola pengurasan akun, yang secara alami lebih mudah terjadi lewat TRANSFER).

4.8 Kesimpulan Phase 4

- Empat metode deteksi (IQR, Z-Score, Isolation Forest, Balance Mismatch) dan satu sinyal cluster (BIRCH+HDBSCAN dari Phase 2) digabung menjadi skor risiko berbobot, dengan bobot lebih besar untuk sinyal multivariat/cluster.
- Sistem berhasil menangkap 17,43% dari seluruh fraud dengan enrichment 17,4x baseline, meski precision tetap rendah (2,24%) karena transaksi bernominal besar tapi sah ikut terflag - risiko umum pada deteksi anomali berbasis statistik murni.
- Ditemukan nuansa penting bahwa skor risiko yang sama bisa berasal dari kombinasi sinyal yang sangat berbeda presisinya (skor 2 justru fraud rate tertinggi, 44,32%) - kombinasi sinyal mana yang aktif sama pentingnya dengan skor totalnya.
- Cross-reference dengan segmen K-Means Phase 2 mengungkap Segmen 4 (Transfer Bernilai Tinggi) sebagai kasus "rare legitimate transaction" yang jelas: sangat menyimpang secara statistik (79% high risk) namun 0% fraud aktual - contoh nyata pentingnya membedakan anomali dari fraud.
- Kategori investigasi otomatis (Potential Fraud, Possible Data Quality Issue, Rare Legitimate Transaction, Potential Risk/Monitor) memberikan kerangka klasifikasi yang langsung menjawab kebutuhan Phase 4 pada dokumen proyek: setiap anomali diklasifikasikan dengan bukti pendukung yang spesifik, bukan sekadar ditandai secara generik.

